

On the Security Risks of Covert Password Manager Providers

Gildas Avoine
INSA Rennes, IRISA

Amit Singh Bhati
KU Leuven, COSIC

Xavier Carpent
Univ. of Nottingham

Diane Leblanc-Albarel
KU Leuven, COSIC

Abstract

Password managers have become widely adopted and are commonly recommended to improve user security. However, current cloud-based solutions rely on the implicit assumption that password manager providers are fully trusted. This assumption is never questioned, as password managers are typically operated by their own designers, who are therefore judge and jury. Users are thus exposed to significant risks: a malicious provider could perform covert actions to access or alter credentials without being detected.

Most password managers rely solely on the strength of a user-chosen master password. This design enables a covert adversary to mount large-scale offline attacks to crack weak master passwords. Even more concerning, some popular password managers do not encrypt credentials on users' devices, transmitting them in plaintext prior to server-side encryption, as is the case with Google default configuration. On the other hand, key-protected password managers such as KeePassXC offer stronger protection but lack native support for multi-device synchronization, limiting their adoption.

Motivated by these observations, we define a comprehensive set of security properties that any cloud-based password manager should satisfy. We demonstrate that none of the widely deployed mainstream solutions fulfill these fundamental requirements. Nevertheless, we argue that it is feasible to design a password manager that is resilient to covert adversaries while supporting secure cross-device synchronization. To support our claims, we propose a provably secure design that meets all identified properties.

1 Introduction

Password managers are services that can be used to secure user credentials, eliminating the need to memorize numerous passwords. Rather, users have a single strong master password that is used to protect all other credentials. These tools are not limited to just storage and have the ability to

generate very secure passwords. Considering these benefits, they are highly encouraged by computer security experts [29], and more so by NIST, when used in conjunction with other controls such as multi-factor authentication and password reuse minimization, i.e., by one authentication, one can access numerous services in a secure manner.

Password managers are available as standalone applications or as browser extensions. For security reasons, the latter are generally discouraged [10]. Regardless of the delivery model, password managers store user credentials in one place, either locally or in the cloud. A wide range of commercial password managers are available, the majority of which are proprietary and rely on cloud-based infrastructures. In such cloud-based systems, the security of stored credentials is critically dependent on the strength of a *master password* chosen by the user and used to encrypt all the other credentials. Thus, if an attacker obtains access to the cloud-stored vault, the security of all stored credentials is reduced to the strength of the user master password. This presents a significant threat, particularly in organizational settings, where it is unrealistic to assume that all employees will consistently select strong master passwords. While locally stored password managers mitigate this risk, they introduce a major limitation: they do not support synchronization across multiple devices. Although organizations may reduce the risk by self-hosting cloud-based password manager solutions, this approach is not widely deployed due to liability issues and does not solve the problem for personal passwords.

Traditionally, a password manager aggregates a set of *credentials*, where each credential typically consists of a service URL, a user identifier, a password, and an optional user-provided note. A group of credentials is stored in a *container*, which may also include metadata for synchronization. The encrypted form of a container is referred to as a *vault*. Vaults are the primary units transmitted from user devices to cloud providers in most modern solutions.

The *password manager provider* (or simply *provider*) refers to the entity that supplies the software both device-side and cloud-side, and operates it cloud-side. For instance, LastPass [23] and Bitwarden [6] are providers that store user vaults in their respective clouds. The *password manager software* is the application or browser extension that users install on their devices to manage accounts. In our context, an *account* (with the provider) holds the container related to an entity (user, group of users, etc.).

Although password managers are widely recommended by security experts, many users still express distrust towards them [13, 18, 14, 17, 25]¹.

¹Studies suggest that this distrust is primarily driven by usability barriers (e.g., the perceived complexity of installing, configuring, and correctly using dedicated tools or browser extensions) and by a risk model in which centralizing credentials is viewed as increasing systemic impact: users perceive the password manager as a high-value single point of compromise whose failure could expose many accounts simultaneously.

Previous research has investigated various dimensions of password managers, including usability [33, 41, 2, 34, 12, 22, 17], password strength [32, 19, 25], data protection [11, 38, 12], and attack surfaces such as auto-fill vulnerabilities and software bugs [21, 11, 32]. In parallel, several alternative architectures have been proposed [7, 37, 39, 8, 40].

In the model used today, where the provider is trusted, several risks are associated with cloud-based password managers. The possibility that a password manager distributes malicious or semi-honest client-side software cannot be excluded. In practice, this risk is partially mitigated by the availability of open-source client implementations that can be independently inspected and audited. Consequently, as in most prior work on password managers, we consider attacks based on malicious client software to be out of scope. Beyond client-side threats, the remaining risks are twofold. First, the provider infrastructure may be compromised by an external adversary, allowing attackers to gain access to vaults stored in the cloud. Second, the security of users' credentials inherently relies on the assumption that the password manager provider itself behaves honestly, i.e., that it does not intentionally misuse, exfiltrate, or covertly access user data despite having privileged control over the service.

Although this trust is commonly accepted (often supported by the public reputation of the provider), such an assumption remains inherently questionable. Providers may act as *covert adversaries* [3], deviating from standard procedures when such deviations are undetectable. For example, a provider could try to crack master passwords to gain access to stored credentials. Various factors could drive such behavior, including pressure from supreme authorities for legal reasons or some competitive intelligence.

It is important to note that some password managers do not implement client-side encryption. Instead, encryption is performed server-side, upon reception of the credentials in the cloud. In such cases, password cracking is not necessary, as the provider already has access to the plaintext credentials prior to encryption. This is, for instance, the default behavior of Google Password Manager[20].

We argue that the assumption of a fully trusted provider is increasingly difficult to justify in realistic adversarial settings. Providers may be compelled by law enforcement or other external actors to access or modify user data. While the Dolev-Yao model [16] is commonly used to analyze protocols², the covert adversary model [3] offers a more realistic framework for evaluating trust assumptions in password manager architectures.

²In this setting, participants (e.g., the user and the provider) are assumed to follow the protocol honestly, while security is analyzed against an active adversary with full control over communications, representing external attackers or other malicious users.

Contributions We challenge the status quo of password manager security by formalizing the service provider as a potential *covert adversary*. We argue that modeling the security of password managers under this covert adversarial model is a worthwhile approach that can lead to a deeper understanding of the real-life risks and challenges associated with these widely used tools. Our specific contributions are threefold:

1. Formal Security Framework: We introduce and formalize a set of five security properties essential for modern credential management. Namely, *Vault Indistinguishability*, *Interaction Type Obliviousness*, *Covert Freshness*, *Covert Integrity*, and *Device Unlinkability*. Unlike prior models, our framework explicitly incorporates resistance to covert deviations, for a better benchmark for verifiable trust in cloud-based services.

2. Analysis of Existing Architectures: We conduct a systematic evaluation of existing password manager schemes (categorized later as Types A, B, and C). Our analysis reveals that current industry standards fail to satisfy several of these critical properties, particularly regarding covert freshness and interaction-type obliviousness, leaving users exposed to undetectable attacks such as rollback attacks.

3. A Verifiably Secure Architecture: To demonstrate that these guarantees are achievable without sacrificing synchronization, we propose a novel password manager protocol. We provide formal cryptographic proofs demonstrating that our protocol maintains user integrity and privacy even in the presence of a covert provider.

The structure of this paper is as follows. Section 2 reviews the state of the art in password manager systems, while Section 3 introduces the necessary technical preliminaries for the modeling of password managers. Section 4 formalizes the cryptographic model and the associated security properties, and evaluates to what extent existing types of password manager satisfy these properties. Our novel password manager architecture is presented in Section 5, which introduces its foundational design, and in Section 6, which provides a complete protocol description. Section 7 provides formal security statements and proof intuitions of how the proposed protocol achieves each property, followed by formal security proofs of these statements in Section 8.

2 State of The Art

In this section, we review the main types of existing password managers. We first provide an overview of the most widely-deployed solutions, then classify them into three main categories. For each category, we analyze the core mechanisms, with particular emphasis on encryption and authentication.

2.1 Commonly Used Password Managers

Password managers store user credentials in one place, either locally on the user device (hereafter referred to as *local password managers*, e.g., KeePassXC[27]) or in the cloud, in which case they are known as *cloud-based password managers*, designed and operated by a password manager provider. The former are significantly less popular, primarily due to the lack of built-in synchronization across multiple devices, and their market share is negligible compared to that of cloud-based solutions. Moreover, as local password managers do not rely on any external provider, the covert adversary model considered in this work does not apply to them. In what follows, we only consider cloud-based password managers. **Done**

Typical cloud-based password manager security analyses assume that the provider is trusted³. This assumption underpins the design of all password managers currently in use. Moreover, from the user perspective, surveys [5] indicate that 54% of worldwide individuals rely on memory and 33% use pen and paper to manage their passwords at home, while only 32% report using a password manager [26, 5]. These results highlight a significant gap between the availability of password manager solutions and their adoption, partly driven by persistent concerns about security and usability among users.

The most widely used password managers [36] are those integrated into platforms provided by major companies such as Google and Apple. Notably, Google Password Manager, despite its widespread usage, has one of the weakest security models: by default, user passwords are sent to Google in plaintext before being encrypted [20], allowing the provider to access any stored credentials. Within the Google Password Manager configuration, users can, after navigating multiple configuration menus, enable client-side encryption instead of the default server-side encryption. Upon enabling this option, the system displays warning messages indicating that loss of the Google account password will result in permanent loss of all stored credentials. Such warnings may discourage most users from opting for client-side encryption despite its stronger security guarantees.

In addition to these built-in solutions, popular third-party services such as LastPass [23], Bitwarden [6], Nordpass [30], Norton [31], 1Password [1], DashLane [15], and RoboForm [35] have emerged. With the exception of 1Password, which augments the user-chosen master password with an additional locally generated secret key, the security of these password managers relies solely on a user-chosen master password, used for authentication, encryption, and decryption.

The security of cloud-based password managers is based on two mechanisms: encryption and authentication. To retrieve a vault, the user must first be authenticated by the provider. In most cases (excluding password man-

³If the password manager is not assumed to be trusted, it reflects the possibility that it may be the target of attacks, rather than an assumption of dishonest behavior.

ager such as Google Password Manager), containers are encrypted locally, and the resulting vault is uploaded to the provider only after encryption. In models such as 1Password, these mechanisms are intentionally separated, with security relying on the additional locally generated key beyond the authentication credential. For all others, both authentication and encryption mechanisms rely exclusively on the master password, although distinct key derivation processes may be used for each function.

2.2 Existing Designs

The security of most password managers rely solely on the strength of the user-chosen master password. When analyzed under a covert adversary threat model the provider may attempt to crack weak master passwords, replay outdated versions of the vault to the user, or collect unencrypted metadata.

To better structure our analysis in Section 4.4, we classify deployed cloud-based password managers into three types.

- **Type A: No Client-Side Encryption.** Password managers in this category, such as Google Password Manager (by default), transmit user data in plaintext to the provider, who performs all encryption server-side.
- **Type B: Password-Based Client-Side Encryption.** This type is the most common. It includes services like LastPass, Bitwarden, Firefox and DashLane. Both authentication and encryption are based solely on a user-defined master password.
- **Type C: Key-Based Client-Side Encryption.** In this type (such as 1Password), encryption requires a combination of a master password and a locally generated cryptographic key. The key is stored on the client device and must be manually transferred to any new device.

In Types B and C, a key derivation function (KDF), such as PBKDF2 or Argon2, is used to derive cryptographic material from the master password and a salt (commonly the user email address). This derivation typically produces two distinct keys: an *authentication key*, used to verify the user identity, and a *key encryption key*, which is used to encrypt a randomly generated *encryption key*. The encryption key is then used to encrypt the container, yielding the vault. The authentication key, the encrypted encryption key, and the encrypted container (vault) are subsequently stored by the provider.

Type C introduces an additional random value called the *secret key*, which is generated locally during account setup. In this case, the key encryption key depends on both the master password and the secret key, thereby

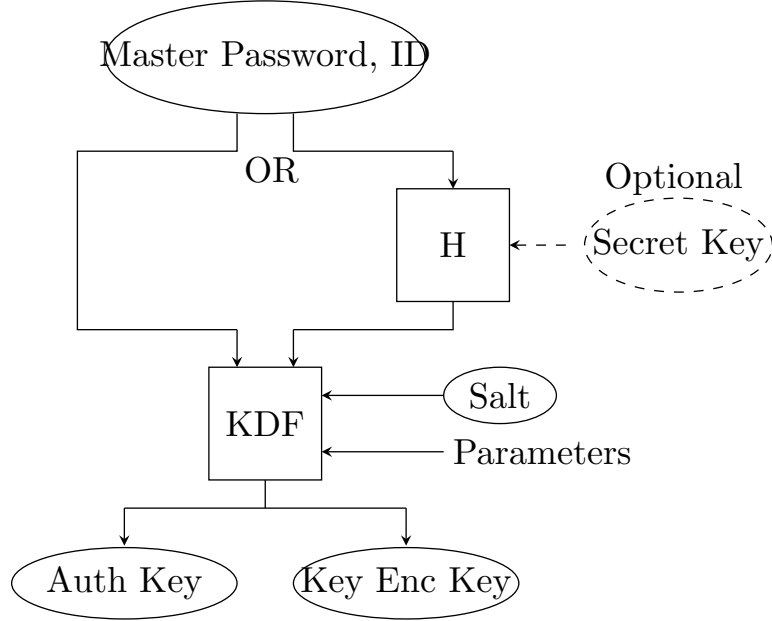


Figure 1: Key Derivation Procedure for Types B and C Password Managers.

significantly strengthening the encryption of the container. Figure 1 illustrates the key derivation process used in Types B and C, highlighting the optional use of a secret key in the latter.

In practice, the number of iterations (also known as *work factor*) used in the key derivation function may differ for the authentication key and the key encryption key, with typical configurations using 600 000 iterations for PBKDF2.

3 Preliminaries

To ensure rigorous analysis of cloud-based password manager protocols, cryptographic primitives and adversarial model are formalized in a common framework for provable security.

3.1 General Notations

We denote the security parameter by $\lambda \in \mathbb{N}$. An algorithm is *efficient* if it runs in probabilistic polynomial time (PPT) with respect to λ . We denote by $\text{negl}(\lambda)$ a negligible function. For a finite set X , $x \xleftarrow{\$} X$ denotes the process of sampling x from X uniformly at random.

Adversary Interaction. We denote an adversary by $\mathcal{A}$ and a challenger by $\mathcal{Ch}$. The interaction between $\mathcal{A}$ and $\mathcal{Ch}$ is modeled through oracle access. The notation $\mathcal{A}^{\mathcal{O}_s} \Rightarrow 1$ describes the scenario where $\mathcal{A}$ makes queries to an oracle $\mathcal{O}$ (without access to $\mathcal{O}$'s internal secret randomness s) and subsequently produces an output of 1.

Advantage. The security of a property *prop* for a protocol Π is quantified by the *advantage* $\text{Adv}_{\Pi}^{\text{prop}}(\mathcal{A})$, which is a measure for $\mathcal{A}$ to successfully violate the said property (e.g., by distinguishing two cases or by deviating undetected), relative to a random guess:

$$\text{Adv}_{\Pi}^{\text{prop}}(\mathcal{A}) = \left| \Pr(\mathcal{A} \text{ succeeds}) - \frac{1}{2} \right|.$$

3.2 Adversary Model: The Covert Provider

The adversary $\mathcal{A}$ in our model is a *covert provider*. Unlike a purely malicious adversary, a covert provider may deviate arbitrarily from the protocol Π , but only if the risk of being caught is negligible.

Definition 1 (Covert Adversary) *A covert adversary $\mathcal{A}$ is a PPT Turing machine representing a provider that may deviate from the prescribed protocol Π . We assume $\mathcal{A}$ prefers to remain undetected; if a deviation can be detected and proven by the users with non-negligible probability, $\mathcal{A}$ is assumed to refrain from such behavior.*

To model realistic scenarios where a provider might already possess some information, we let $\tilde{\mathbf{A}} \subset \mathbf{A}$ represent a strict subset of accounts (out of all accounts $\mathbf{A}$) to which $\mathcal{A}$ has full access (including vaults, containers, passwords, and keys). Security is therefore analyzed based on the accounts in $\mathbf{A} \setminus \tilde{\mathbf{A}}$.

3.3 Cryptographic Primitives

We define the formal syntax of the Authenticated Encryption and Digital Signatures used in the our constructions.

Authenticated Encryption (AE) An AE scheme is a triple of algorithms $\Sigma = (\text{KGen}, \text{Enc}, \text{Dec})$ such that:

- $\Sigma.\text{KGen}(1^\lambda) \rightarrow K$: Generates a symmetric key K .
- $\Sigma.\text{Enc}_K(M) \rightarrow C$: Encrypts message M with key K to produce ciphertext C .
- $\Sigma.\text{Dec}_K(C) \rightarrow M \cup \{\perp\}$: Decrypts C ; returns $\perp$ if integrity is violated.

Digital Signature Scheme. A signature scheme is a triple of algorithms $\mathcal{S} = (\text{Gen}, \text{Sign}, \text{Vrfy})$ such that:

- $\mathcal{S}.\text{Gen}(1^\lambda) \rightarrow (pk, sk)$: Generates public/private key pair.
- $\mathcal{S}.\text{Sign}_{sk}(M) \rightarrow \sigma$: Produces signature σ on message M .
- $\mathcal{S}.\text{Vrfy}_{pk}(M, \sigma) \rightarrow \{0, 1\}$: Outputs 1 iff signature is valid.

We require Σ to provide *IND-CCA security* (Indistinguishability under Adaptive Chosen Ciphertext Attack) and *INT-CTXT security* (Ciphertext Integrity) [4], and $\mathcal{S}$ to provide *EUFCMA security*⁴ (Existential Unforgeability under Chosen Message Attack) [4]. Definitions for these properties is quite standard and are reminded in Appendix 10.

4 Password Manager: Syntax and Properties

This section details the assumptions and properties that are expected from a cloud-based password manager to be resistant to covert adversaries. The (non-)resistance of existing solutions against this threat model is also discussed.

4.1 System Components and Interactions

In order to model the interactions and security properties of a cloud-based password manager, we introduce its components and their interactions, depicted in Figure 2.

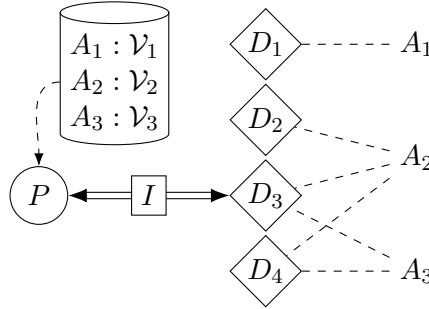


Figure 2: Components and their interactions. Provider P stores vaults $\mathcal{V}_i$ for each account A_i , and communicates using interaction I with devices D_j .

Accounts A_j are user accounts. The key K_{A_j} is the encryption key associated with A_j , and can be randomly generated or derived from a master password. Each account is associated with a *container* $\mathcal{C}_j$ (which contains

⁴EUFCMA is the common abbreviation, however, in [4] it is referred as WUF-CMA.

credentials, along with associated metadata), and a *vault* $\mathcal{V}_j$ which is an encrypted version of $\mathcal{C}_j$ with key K_{A_j} .

The *provider* P is the password manager provider. It stores vaults of accounts registered with it.

Devices D_i are users' devices. Accounts can be set up on multiple devices, and devices can be set up with multiple accounts. For every account, devices associated to it keep a copy of the vault. Devices are supplied with software to run the password manager service and the interactions with P . We consider that this software is not malicious, and in particular that it behaves according to the defined protocol, and does not leak data through side channels.

Finally, P interacts with devices in an *interaction* I . Each interaction has a *type* T_I . We denote the set of possible interaction types $\mathcal{T}$. For example, this set might be $\mathcal{T} = \{\text{READ}, \text{UPDATE}\}$, where:

- **READ:** D fetches the newest vault, no update
- **UPDATE:** D fetches the newest vault, the container is updated and the new resulting vault sent to P

4.2 Protocol Syntax

A cloud-based password manager scheme Π consists of the following algorithms:

- **Setup**(1^λ) $\rightarrow (K_{ID}, \text{ID})$: Executed by the user to generate a master key and account identifier.
- **Enc** $_{K_{ID}}(\mathcal{C}) \rightarrow \mathcal{V}$: Encrypts a container $\mathcal{C}$ into a vault $\mathcal{V}$.
- **Dec** $_{K_{ID}}(\mathcal{V}) \rightarrow \mathcal{C} \cup \{\perp\}$: Decrypts a valid vault.
- **Interact**(D, P, T_I) $\rightarrow \text{trans}$: A protocol between device D and provider P for a specific interaction type $T_I \in \mathcal{T}$.

Here **trans** denotes the *protocol transcript*, which is the ordered sequence of all messages exchanged between the device D and the provider P during an execution of **Interact**. In our security model, **trans** represents the complete information available to the provider, and thus constitutes the *view* of the adversary $\mathcal{A}$ during the protocol execution.

4.3 Security Properties

We now formalize the 5 security properties that any cloud-based password manager scheme Π should guarantee: *Vault Indistinguishability*, *Interaction Type Obliviousness*, *Covert Freshness*, *Covert Integrity*, and *Device Unlinkability*. For that, adversary $\mathcal{A}$ is used to represent the (covert) provider P .

4.3.1 Vault Indistinguishability

Π must guarantee that no information about a container $\mathcal{C}$ can be extracted from its vault $\mathcal{V}$ (semantic security). Furthermore, P should be unable to distinguish between the encryptions of 2 containers (Indistinguishability). The latter implies semantic security, so it captures the meaningful properties on its own. The following security game captures this notion.

Property 1 $\mathcal{A}$ chooses two distinct containers $\mathcal{C}_0$ and $\mathcal{C}_1$, and an account $A \notin \tilde{\mathbf{A}}$, and submits them to a challenger Ch . Ch then picks $b \in \{0, 1\}$ uniformly at random, encrypts $\mathcal{V} = \Pi.\text{Enc}_{K_A}(\mathcal{C}_b)$, and sends $\mathcal{V}$ to $\mathcal{A}$. $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$. The scheme provides Vault Indistinguishability iff for all efficient $\mathcal{A}$ s:

$$\text{Adv}_{\Pi}^{v\text{-ind}}(\mathcal{A}) := \left| \Pr(b = b') - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

4.3.2 Interaction Type Obliviousness

The interaction I between devices and the provider should not reveal any information about modifications performed on the account. In particular, this means that Π must guarantee that P cannot determine whether the container has changed.

Property 2 $\mathcal{A}$ picks an account $A \notin \tilde{\mathbf{A}}$. Ch then selects a device D uniformly at random among those belonging to A . Ch also picks $T \in \mathcal{T}$ uniformly at random. $\mathcal{A}$ and D then interact using I s.t. $T_I = T$. $\mathcal{A}$ outputs a guess $T' \in \mathcal{T}$. The scheme is Interaction-Type-Oblivious iff for all efficient $\mathcal{A}$ s:

$$\text{Adv}_{\Pi}^{it\text{-obl}}(\mathcal{A}) := \left| \Pr(T = T') - \frac{1}{|\mathcal{T}|} \right| \leq \text{negl}(\lambda).$$

4.3.3 Covert Freshness

P should issue the latest version of the vault in each interaction between devices. The *latest vault* for each account is defined as the vault that was last received by P from any device of the relevant account. The scheme must allow account-holders to detect deviations from this.

Property 3 $\mathcal{A}$ picks an account $A \notin \tilde{\mathbf{A}}$. Ch then selects a device D uniformly at random among those belonging to A . Let $\mathcal{V}^{(1)}, \dots, \mathcal{V}^{(n)}$ be the series of historic vaults produced by the account A . Vault $\mathcal{V}^{(n)}$ is the latest vault. Let $\mathcal{V}^{(j)}$ be the vault provided by $\mathcal{A}$ to D in an interaction I . The scheme Π guarantees Covert Freshness with deterrence threshold δ iff for all efficient $\mathcal{A}$ s:

$$\text{Adv}_{\Pi}^{c\text{-fr}}(\mathcal{A}) := 1 - \Pr[\text{Ch detects misbehavior} \mid j \neq n] \leq (1 - \delta).$$

Misbehavior detection implies verifiable evidence (e.g., a cryptographic proof or inconsistency in authentication tags, timestamps, etc.).

Ideally, $(1-\delta) \leq \text{negl}(\lambda)$ or $\delta \approx 1$ would be expected to ensure deterrence. However, this is unattainable in distributed settings without synchronous communication between devices. Now, with the help of game theory, we argue that any non-negligible value of δ (e.g., $\delta = 1/100$) is equally good as long as the covert provider's expected utility is negative, i.e., $(1-\delta) \cdot \text{Gain} < \delta \cdot \text{Cost}$.

In simple words, the *Cost* of detection (e.g., total loss of reputation, legal liability, or business termination) is typically orders of magnitude higher than the *Gain* from a single account rollback. Consequently, a significant non-negligible probability such as $\delta = 1/d$ (where d is the maximum number of devices that can be associated with an account) provides a robust practical deterrent when $d < 1 + \text{Cost}/\text{Gain}$, as the provider cannot predict if the device with the latest vault is the one reconnecting and thus faces a constant, non-trivial risk of being caught with undeniable evidence (See Appendix 9 for details). We can thus update the advantage bound of $\mathcal{A}$ against a C-FR secure scheme Π with $d < 1 + \text{Cost}/\text{Gain}$ as

$$\text{Adv}_{\Pi}^{\text{c-fr}}(\mathcal{A}) \leq \left(1 - \frac{1}{d}\right).$$

4.3.4 Covert Integrity

Π must guarantee that, when provided with the legitimate latest container $\mathcal{C}$ (i.e., matching the latest vault) of an uncompromised account, $\mathcal{P}$ cannot modify $\mathcal{C}$ without being detected (even if respecting Covert Freshness).

Property 4 *$\mathcal{A}$ who respects Covert Freshness picks an account $A \notin \tilde{\mathbf{A}}$. $\mathcal{Ch}$ then selects a device D uniformly at random among those belonging to A . Let $\mathcal{C}^{(n)}$ be A 's latest container. Let $\mathcal{V}$ be the vault provided by $\mathcal{A}$ in an interaction I with D , and $\mathcal{C} := \Pi.\text{Dec}_{K_A}(\mathcal{V})$. The scheme Π guarantees Covert Integrity iff for all efficient $\mathcal{A}$ s:*

$$\begin{aligned} \text{Adv}_{\Pi}^{\text{c-int}}(\mathcal{A}) &:= 1 - \Pr[\mathcal{Ch} \text{ detects misbehavior} \mid \mathcal{C} \neq \mathcal{C}^{(n)}] \\ &\leq \text{negl}(\lambda). \end{aligned}$$

Misbehavior detection implies verifiable evidence (e.g., a cryptographic proof or inconsistency in authentication tags, timestamps, etc.).

4.3.5 Device Unlinkability

Given multiple interactions belonging to the same account, the provider should not be able to determine if they originated from the same device or

different devices. This property, at the interaction level, allows an account to hide from a covert provider, the number and usage of devices associated to it.

Property 5 *$\mathcal{A}$ chooses an account $A \notin \tilde{\mathbf{A}}$ (which is linked with at least two devices) and two interaction types T_{I_1} and T_{I_2} . Ch picks two devices D_0 and D_1 uniformly at random among those belonging to A . Ch then picks $b \in \{0, 1\}$ uniformly at random. Then D_0 and D_b interacts with $\mathcal{A}$ using interaction types T_{I_1} and T_{I_2} , respectively. Finally, $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$. The scheme provides Device Unlinkability iff for all efficient $\mathcal{A}$:*

$$\text{Adv}_{\Pi}^{d\text{-}unlk}(\mathcal{A}) := \left| \Pr(b = b') - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

Note that we need each account to be linked with at least two devices. However, the definition also captures that $\mathcal{A}$ cannot distinguish A from accounts associated with a single device. The property thus applies to all accounts associated with at least one device.

4.3.6 Devices and Accounts Associations

P should have no knowledge of the bipartite relationship graph between accounts and devices, with the exception of the knowledge of how many accounts it manages. In particular, P should have no knowledge of how many devices there are in the system, nor how many (or indeed, which) devices each account is linked to. This property is obtained, at the protocol level by the combination of *Device Unlinkability*, *Vault Indistinguishability*, and *Interaction Type Obliviousness*.

4.4 Security of Existing Approaches

As stated in Section 2, we categorize existing cloud-based password managers into three types. The first one, referred to as *Type A*, includes password managers that perform no client-side encryption: encryption is handled by the provider. Google Password Manager, used by a majority of users [36], is a typical example. The second category, *Type B*, corresponds to password managers implementing the encryption scheme described in Figure 1, where both authentication and encryption rely on derivation of the same master password. This category includes the majority of currently deployed managers, such as LastPass, Bitwarden, DashLane, and Firefox. Finally, *Type C* are password managers that rely on both a master password for authentication and a device-specific cryptographic key for encryption, as used by services such as 1Password.

We discuss, and summarize in Table 1, the resistance of each type (A, B, C) with respect to the 5 considered properties.

4.4.1 Type A: No Client-Side Encryption

The provider has access to all containers and the corresponding vaults. Consequently, Properties 1 to 4 are clearly not satisfied. Property 5 (*Device Unlinkability*) may or may not be satisfied, depending on implementation. For example, Google stores a device identifier and asks for user confirmation when a new device attempts to retrieve passwords. However, this behavior is not inherent to Type A, but rather a design choice.

4.4.2 Type B: Password-Based Client-Side Encryption

Property 1 (*Vault Indistinguishability*) is generally not satisfied by *Type B*, as the vault size often leaks information about the container. More importantly, weak master passwords may be guessed by a provider. However, this limitation stems from implementation details rather than the model itself. In principle, Property 1 is achieved only for accounts with strong master passwords. As, in practice, the encryption IV is the user’s email address, Property 2 (*Interaction Type Obliviousness*) is not satisfied: vaults remain unchanged when container and master password are unchanged. Property 3 (*Covert Freshness*) is not satisfied, as providers can replay outdated vaults. Property 4 (*Covert Integrity*) is only satisfied if the master password is strong and if an AE is used, which never happens in practice. Property 5 (*Device Unlinkability*) is often violated due to device tracking (e.g., Bitwarden stores a “device-specific GUID”), but this comes from a design choice, rather than a limitation of *Type B*.

4.4.3 Type C: Key-Based Client-Side Encryption

The encryption relies on a strong cryptographic key stored on the user’s device in addition to a master password. Synchronization is less convenient, requiring the user to manually transfer the key to each new device. Property 1 may be satisfied if the provider stores no metadata and vault sizes are independent from container sizes. Otherwise, Property 1 does not hold. Properties 2 and 3 are not satisfied for the same reasons as in Type B. Property 4 (*Covert Integrity*) is guaranteed due to the strong cryptographic key used for container encryption. Property 5 (*Device Unlinkability*) may be satisfied depending on the implementation as for Type A and B.

5 Password Manager Setup

After showing that none of the three types of password managers can guarantee all considered properties, we demonstrate that it is nevertheless possible to design one that satisfies them. This section presents the foundations of our approach, while the core protocol is described in Section 6.

Table 1: Security Property of Deployed Password Managers

Property / Type	A	B	C
Vault Indistinguishability	○	◐ ◑	◐
Interaction Type Obliviousness	○	○	○
Covert Freshness	○	○	○
Covert Integrity	○	◐	●
Device Unlinkability	◐	◐	◐

- Property not guaranteed.
- ◐ Property only guaranteed for accounts with strong master passwords.
- ◑ Property not guaranteed by most services, but this is due to implementation choices, not architectural limitations.
- Property guaranteed.

5.1 Account Creation

To create an account, the provider supplies the user with a random account identifier ID and an empty container $\mathcal{C}_{\text{ID}}$.

A prerequisite for Property 1 (*Vault Indistinguishability*) is for all vaults to have the same size – otherwise the size itself could be a distinguisher. Two distinct containers can actually differ in size (e.g., different number of credentials, longer passwords, URLs, notes, etc.) but when encrypted, they are padded such that the resulting vaults have equal size, denoted L in what follows.

This restriction also caps container sizes, which may be undesirable. This introduces a trade-off: a larger L enables larger containers but increases overhead, so a sensible L should be chosen⁵. Ultimately, this decision rests with the provider, and many reasonable options exist. For example, $L = 256$ KB is small enough to limit the impact on resources, yet sufficient for 1000 credentials containing a username and password, a 100-character URL, an 80-character note, and up to ten 4-byte timestamps. This is a conservative estimate, as containers are likely to benefit from compression.

5.2 Cryptographic Material

The key-point of our design is to associate each account with a structured container that stores not only user credentials but also additional meta-data necessary for ensuring proper synchronization and detecting malicious behavior from the provider.

⁵Other compromises are also possible, one can for instance slightly relax the Vault Indistinguishability property by introducing several possible sizes $L_1, L_2, L_3, \dots$ that users can opt into (this provides k -indistinguishability).

Along with the unique identifier ID, each account is associated with an encryption key K_{ID} . Indeed, unlike traditional approaches where containers are encrypted using a master password, our solution employs a dedicated cryptographic key K_{ID} for each account. This key is randomly generated by the device that creates the account and remains unknown to the provider. The provider only learns the number of accounts it manages, but has never access to any other information such as the number of devices, the number of users, or any information about contents of containers.

Multiple devices can be granted access to a single account. If several devices (possibly belonging to different users) are authorized to access a given account, they all share the same identifier ID and the same encryption key K_{ID} associated with that account.

Each device also has a master password for every account it is authorized to access⁶. When there is no ambiguity, meaning that a single device and a single account is considered, the master password is denoted p^* . As explained below, p^* is only used to encrypt information that is stored locally, namely the encryption key K_{ID} . As stated in Section 2, encrypting containers with a key derived from the master password is commonly used by cloud-based password managers, although it is highly insecure when considering a covert provider.

5.3 Container Structure

The container is the main structure associated with a password manager account. It is where all credentials are stored. An account identified by ID is thus associated with a container $\mathcal{C}_{ID}$. A key aspect of our design is the inclusion of a timestamp list for each credential p_i ($1 \leq i \leq n$), as illustrated in Figure 3. Indeed, the container is a structured file that contains the identifier ID, a synchronization counter w representing the number of successful interactions with the provider for that account, and a list of credentials along with timestamps. A credential p_i ($1 \leq i \leq n$) consists of a password and optional information (username, URL, notes etc.).

Figure 3 illustrates a container $\mathcal{C}_{ID}$ that contains n credentials $p_1, \dots, p_n$, shared by m devices $D_1, D_2, \dots, D_m$, represented by m columns containing timestamps. Each device stores such a container (in an encrypted form), so we denote $\mathcal{C}_{ID}^{D_1}, \mathcal{C}_{ID}^{D_2}, \dots, \mathcal{C}_{ID}^{D_m}$, the copy of the container on each device. When there is no ambiguity, we simply use $\mathcal{C}_{ID}^D$. Once encrypted, the container is called a vault, and we consequently respectively denote the copy of the vault on each device as $\mathcal{V}_{ID}^{D_1}, \mathcal{V}_{ID}^{D_2}, \dots, \mathcal{V}_{ID}^{D_m}$ ($\mathcal{V}_{ID}^D$ when no ambiguity). The copy of the vault stored by the provider, P , is denoted as $\mathcal{V}_{ID}^P$.

⁶For the sake of usability, a user can use the same master password for all the accounts and devices it is authorized to access to. Although not recommended, this approach does not significantly decrease the security level, given that the master password is used locally only.

Given a container $\mathcal{C}_{\text{ID}}^{D_k}$ ($1 \leq k \leq m$), $t_{i,j}^{D_k}$ represents D_k 's knowledge about the last time D_j modified p_i , e.g., $t_{4,2}^{D_1} < t_{4,2}^{D_2}$ implies that the credential p_4 stored by D_1 is obsolete.

ID				
w				
p_1	$t_{1,1}$	$t_{1,2}$	$\dots$	$t_{1,m}$
p_2	$t_{2,1}$	$t_{2,2}$	$\dots$	$t_{2,m}$
p_3	$t_{3,1}$	$t_{3,2}$	$\dots$	$t_{3,m}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
p_n	$t_{n,1}$	$t_{n,2}$	$\dots$	$t_{n,m}$

Figure 3: Container Structure

5.4 Vault Storage

While a *container* $\mathcal{C}_{\text{ID}}^D$ is a plaintext structure that should never be stored in its raw form, a *vault* $\mathcal{V}_{\text{ID}}^D$ is a randomized encryption of $\mathcal{C}_{\text{ID}}^D$ with K_{ID} . It is worth reminding that K_{ID} is not derived from a password, but randomly generated. Note that the vault should be encrypted using the AES-GCM authenticated encryption (AE) scheme, which provides both confidentiality and data integrity.

Locally, each device stores a copy of the vault (it stores only the most recent version it knows) along with the account identifier ID and the encryption of the key $E_{p^*}(K_{\text{ID}})$ where E is a password-based encryption function.

Remotely, the provider also stores the most recent copy of the vault it knows. It must be able to retrieve it correctly from its storage structure when a user requests it with ID. Thus it may store ID or the hash of ID, along with the vault.

5.5 Key-exchange Between Devices

We assumed so far that the devices associated to a given account share a common secret key K_{ID} to protect the vault. However, we have not yet described how these devices establish this shared key.

When a new device joins an existing account, it must obtain the key K_{ID} from an already configured device of that account. This can be viewed as a sponsorship. This task requires both devices to be online simultaneously to securely exchange K_{ID} .

For that, we recommend using a key exchange protocol with authentication relying on short secrets to optimize user experience without sacrificing

security. Once the shared key is established, it enables a secure messaging channel through which the password manager key can be safely transmitted.

Various well-established techniques exist in the literature for setting up such a secure channel, avoiding to design a new one. For instance, the Numeric Comparison key-exchange protocol, originally designed for Bluetooth, meets all the necessary requirements. This protocol, based on Elliptic Curve Diffie-Hellman (ECDH), is compatible with both Bluetooth Classic and Bluetooth Low Energy. It allows an Initiator and a Responder to securely exchange a key, with authentication relying on short secrets: both devices display a 6-digit code, and the user verifies that they match before confirming by pressing a button on each device. The 6-digit code could be replaced with a more secure and user-friendly approach, such as using emojis. Note that making these protocols PQ-resistant is already considered [24].

Although Bluetooth can be used to transmit the password manager secret key, the Numeric Comparison key-exchange protocol itself is transport-agnostic. It can be implemented over any communication channel, provided the user can see both device displays to confirm authentication.

6 Password Manager Protocol

This section presents a detailed description of the proposed password manager protocol. Its correctness is then analyzed.

6.1 Protocol Description

Figure 4 depicts the protocol to be used between a device and the provider. It is atomic, meaning it is designed to be executed entirely. Its execution is blocking, meaning that the provider should not run two instances of the protocol in parallel for a given account. In practice, provider-side and device-side timeouts can avoid the parties from being stuck in case of failure. The actions presented in Figure 4 are described below.

Identify (first message). A device sends to the provider the identity ID of the account to which it requests access.

Pull (second message). Upon receiving a request from a device, the provider sends back the vault, $\mathcal{V}_{\text{ID}}^P$, which it currently stores for the requested ID.

Check. This procedure verifies that the received vault $\mathcal{V}_{\text{ID}}^P$ is consistent with the locally stored vault $\mathcal{V}_{\text{ID}}^D$ on the device, and, if necessary, performs a resynchronization. This procedure plays a critical role and is described in detail in Section 6.2.

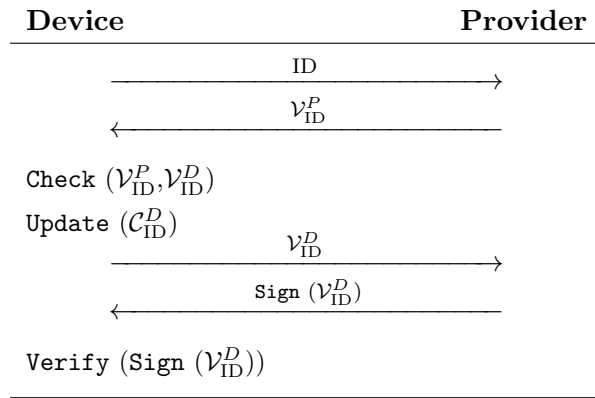


Figure 4: Exchanges between a device and the provider

Update. When a device D modifies credentials in its container (temporary local copy), it must also increment the counter w and the timestamps of the credentials considered (only in its column). Whether or not the credentials have been modified, the container $\mathcal{C}_{ID}^D$, is re-encrypted (randomized encryption) before being pushed to the provider. The update remains pending until the protocol successfully ends.

Push (third message). A device pushes its vault to the provider.

Sign (fourth message). The provider must acknowledge the reception of the vault. To do so, it signs the received vault and sends the signature to the device. It is important to note that the signature must be undeniable.

Verify. The device checks that the signature sent by the provider is valid w.r.t. the vault it locally stores. If so, the device definitively replaces $\mathcal{V}_{ID}^D$ by the updated one.

6.2 Check Procedure

Because the **Check** procedure is a core component of the protocol and one of the main differences from existing managers, and because its structure and its impact on the guarantees of Property 3 depend on the number of devices, we detail it for both the single-device and the multi-device settings.

6.2.1 Single Device

When there is a single device D associated to an account, the **Check** procedure is trivial. Upon reception of $\mathcal{V}_{ID}^P$, the device decrypts it using K_{ID} and obtains $\mathcal{C}_{ID}^P$. The device D decrypts the locally stored vault, obtaining $\mathcal{C}_{ID}^D$.

If the two containers $\mathcal{C}_{\text{ID}}^P$ and $\mathcal{C}_{\text{ID}}^D$ perfectly match, the **Check** procedure is successful.

6.2.2 Several Devices

When a single user accesses a password manager account from several devices or when several users (with different devices) share a common account, then several devices ($D_1, D_2, \dots, D_m$) may access and modify the shared vault⁷. In this highly likely scenario, devices may have inconsistent versions of the vault. We describe in the following how the synchronization is maintained using the **Check** procedure, when the protocol is executed between a device, e.g., D_1 , and the provider P :

1. D_1 decrypts the locally stored $\mathcal{V}_{\text{ID}}^{D_1}$ and obtains $\mathcal{C}_{\text{ID}}^{D_1}$.
2. D_1 decrypts the received $\mathcal{V}_{\text{ID}}^P$ and obtains $\mathcal{C}_{\text{ID}}^P$.
3. D_1 verifies that $w_{\text{ID}}^P \geq w_{\text{ID}}^{D_1}$, where w_{ID}^P (resp. $w_{\text{ID}}^{D_1}$) is the value of the synchronization counter of the account (identified by ID) as known by P (resp. D_1). If so, the process continues, otherwise, an alert is raised, meaning that $\mathcal{V}_{\text{ID}}^P$ as sent by P is older than the one previously seen by D_1 .
4. D_1 checks that the timestamps contained in column D_1 of $\mathcal{C}_{\text{ID}}^P$ match the ones of $\mathcal{C}_{\text{ID}}^{D_1}$.
5. D_1 updates the container timestamps to correct any desynchronization.

The case where D_1 is desynchronized occurs, for instance, if D_1 modified credentials off-line, without running the protocol, while another device D_2 modified credentials and ran the protocol in the meantime. This scenario requires to resynchronize the containers, which consists for D_1 to update each timestamp $t_{i,j}^{D_1}$ with $\text{Max}(t_{i,j}^{D_1}, t_{i,j}^P)$.

Note that D_1 and P are resynchronized once the protocol ends (without failure), but D_2 will only be resynchronized after its next execution of the protocol.

6.2.3 Practical Considerations

Below, we discuss some practical aspects of our protocol. More detailed considerations are provided in Appendix 9.

⁷In that case, the provider has no means of distinguishing between one user with multiple devices and several users sharing a common account (e.g., in organizational settings or family account sharing).

The **Check** procedure, automatically executed by the devices is necessary to ensure Property 3 (*Covert Freshness*). Specifically, this procedure enables a device to detect a misbehaving provider that supplies an outdated vault.

However, immediate detection is not guaranteed when multiple devices are authorized to access the same account. Indeed, the **Check** procedure performed by a device allows it to detect that the vault provided by the server is outdated if one of the devices already accessed a more recent version of the vault. This detection is based on the synchronization counter w , which enables a device to identify regressions relative to its own local history.

Hence, if the provider delivers to a device an outdated version of a vault that is still consistent with the device’s local history and has not yet been flagged as outdated by any other device, the device cannot detect the desynchronization unless synchronous communication between devices is assumed.

As discussed in Section 4.3.3, the provider cannot distinguish among the devices with which it interacts and therefore cannot predict whether a deviation will be detected by the **Check** procedure. Thus, under the covert adversary model, the provider must assume that sending an outdated vault may be detected, and should therefore refrain from doing so.

It is worth noting that setting up a synchronous channel between devices makes sense in some real-world scenarios, when the channel is human-based. In that case, the user manually verifies that the received vaults are consistent with those previously observed on other devices. Concretely, the user is asked to verify that the synchronization counter w of the received vault matches the last value it pushed from another device. Given that requiring users to memorize a numeric value may be cumbersome, the process could be made more user-friendly using techniques from the field of Human-Computer Interaction, e.g., verification based on images.

As a final remark, our protocol is secure under the covert provider model, even when the communication channel between the devices and the provider is insecure. Nevertheless, we conservatively recommend protecting these communications using a secure channel, e.g., TLS. This practice provides defense against external adversaries. Establishing such a channel is a common practice that is not described in this paper.

6.3 Protocol Correctness

A cloud-based password manager is a distributed service involving multiple devices that share a common vault. Ensuring the synchronization is of the utmost importance. Our proposal guarantees that, if there is no attack, any device that successfully performs the protocol obtains the most up-to-date version of the vault uploaded so far onto the cloud. To prove the protocol correctness, we assume that all parties act correctly – meaning that **Check**, **Update**, **Sign**, and **Verify** are performed according to their

specifications – and the parties do not lose data. We analyze the only two cases where desynchronization can arise : (i) some messages are lost, (ii) some offline devices update their credentials locally.

6.3.1 Lost Messages

We remind that when a device modifies its container, it works on a temporary copy of it, and it eventually updates it once and only once the protocol is successfully executed.

The case where there is a single device associated to the considered account is trivial: if a message is lost, the device detects that the protocol did not end successfully (given that the last message goes from the provider to the device), and consequently it does not update its container. So, a device alone cannot be desynchronized.

In what follows, we consider the case where there are at least 2 devices sharing a common password manager account. For clarity, we denote m_1 , m_2 , m_3 , and m_4 the 4 messages (from top to bottom) of the protocol presented in Figure 4.

- If m_1 or m_2 is lost, there is no desynchronization because the device and the provider wait for m_3 before replacing their local copy of the vault.
- If m_3 is lost during the execution of the protocol between a device D and the provider P , the vault hosted by P remains unchanged. Obviously, P does not send m_4 , and the vault hosted by D thus remains unchanged: there is no desynchronization.
- If m_4 is lost, D does not replace its vault by the temporary one, while P replaces the vault it hosts with the received one in m_3 . The consequence is that any device that runs the protocol with P will receive the fresh vault while D remains with the old one. In such a scenario, D detects that the protocol did not end properly, since m_4 is missing, and thus re-runs the protocol until receiving m_4 . Synchronization is then eventually restored.

6.3.2 Offline Devices

The second scenario that can lead to desynchronization is when offline devices modify credentials. The case where a single device is associated with the considered account is trivial. We so consider at least two devices in the following.

We assume that a device D successfully performs the protocol at some point. Later, D modifies some credentials locally, and then executes the protocol again. We dissect this second execution of the protocol below to prove that no desynchronization can arise. Two cases are analyzed below.

- If no device modifies the container between two protocol executions, as detected by D (i.e., between the end of the first execution and the start of the second), then D simply uploads its updated vault. Since protocol executions are blocking, no other device can upload a new vault during this interval, and no desynchronization can occur.
- If some devices modified the container between the two executions of the protocol, which is detected by D during the **Pull** operation, then D first updates its local copy of the container as defined in Section 6.1, adds its modifications, and executes the **Push** procedure. Other devices will be resynchronized once they execute the protocol.

7 Protocol Security

This section provides security intuition and arguments explaining why our protocol, hereafter referred to as $\Pi^\dagger$, satisfies all the properties defined in Section 4.3, we then formally prove that it is secure against a covert adversary $\mathcal{A}$ in Section 8.

Protocol Instantiation. To enable formal analysis, we specify that the protocol $\Pi^\dagger$ is instantiated using two primary cryptographic components defined in Section 3.3: (1) an AE scheme $\Sigma = (\text{KGen}, \text{Enc}, \text{Dec})$ used for all vault encryption and decryption operations, and (2) a digital signature scheme $\mathcal{S} = (\text{Gen}, \text{Sign}, \text{Vrfy})$ used by the provider to sign vault updates. More formally, $\Pi^\dagger.\text{Enc}_K(M) := \Sigma.\text{Enc}_K(M)$ and $\Pi^\dagger.\text{Dec}_K(M) := \Sigma.\text{Dec}_K(M)$.

7.1 Vault Indistinguishability

Theorem 1 (V-IND) *If Σ is IND-CCA secure and all containers are padded to a constant length L , then protocol $\Pi^\dagger$ satisfies Vault Indistinguishability.*

Intuition: Since the provider $\mathcal{A}$ does not possess the randomly generated key K_{ID} , they act as an eavesdropper against the vault. The IND-CCA security of Σ ensures that the ciphertext (vault) hides all information about the plaintext (container). Also, the fixed-length padding L ensures that the provider cannot use the vault size as a side-channel to distinguish a container with many credentials from one with fewer.

7.2 Interaction Type Obliviousness

Theorem 2 (IT-OBL) *Protocol $\Pi^\dagger$ provides Interaction Type Obliviousness if it satisfies Vault Indistinguishability and the device-side execution of **Check** and **Update** is constant-time.*

Intuition: The provider observes a sequence of messages that are structurally identical for both READ and UPDATE interactions. Because Σ is a randomized encryption scheme, a fresh encryption of an unchanged container (in a READ) is computationally indistinguishable from an encryption of a modified container (in an UPDATE); a property captured under $\Pi^\dagger$'s Vault Indistinguishability. By enforcing constant-time execution of the protocol steps **Check** and **Update**, we ensure the provider cannot use the duration of the device's processing as a signal to identify the interaction type.

7.3 Covert Freshness

Theorem 3 (C-FR) *If $\mathcal{S}$ is EUF-CMA secure and the protocol $\Pi^\dagger$ satisfies Device Unlinkability, then it also satisfies Covert Freshness with a deterrence threshold $\delta = 1/d$, where d is the maximum number of devices that can be connected to an account.*

Intuition: This property relies on the digital signature $\mathcal{S}$ acting as a *non-repudiable receipt*. When a device pushes an updated vault with the new synchronization counter w , the provider is required to sign it and send the signature back to the device. The EUF-CMA assumption ensures that no one can forge provider signature, making every provider signature non-repudiable. If the provider later attempts to serve an old vault $\mathcal{V}^{(w-k)}$, any device that has already seen version w will detect the regression. Because the provider cannot identify which device is connecting (due to unlinkability) and does not know the number of devices d , it must “gamble” on whether the device is outdated; if they guess wrong (which has probability $\geq 1/d$, as at least one device is up-to-date), the user obtains verifiable evidence of the provider misbehavior.

7.4 Covert Integrity

Theorem 4 (C-INT) *Protocol $\Pi^\dagger$ satisfies Covert Integrity if Σ is INT-CTXT secure.*

Intuition: While Freshness prevents the use of old data, Integrity prevents the modification of current data. The INT-CTXT property (Integrity of Ciphertext) of Σ ensures that any unauthorized modification or bit-flip performed by the provider on the vault will cause the device decryption operation to fail (returning $\perp$). In our protocol, this failure constitutes immediate detection of provider-side tampering.

7.5 Device Unlinkability

Theorem 5 (D-UNLK) *Protocol $\Pi^\dagger$ satisfies Device Unlinkability if it satisfies Vault Indistinguishability and the device-side execution is normalized to constant absolute time.*

Intuition: Hardware performance varies across device classes (e.g., a high-performance PC vs. a low-power IoT device). To prevent the provider from “fingerprinting” a device based on its processing speed, the protocol requires devices to normalize their response times. By ensuring that the device responds only after a fixed duration T_{norm} (where T_{norm} is greater than the worst-case processing time of the slowest supported device), the absolute timing side-channel is closed, rendering the interactions from a PC and an IoT device indistinguishable to the provider (See Appendix 9 for details).

The provider view of the protocol now consists only of the account ID and the encrypted vaults. Because $\Pi^\dagger$ is V-IND secure, the vaults are indistinguishable from random noise for the provider. Since the protocol does not include device-specific nonces, timestamps, or metadata in the plaintext headers, there is no information in the transcript that allows the provider to link two separate sessions to the same physical device.

8 Security Proofs

We provide below formal proofs for Theorem 1-5.

Proof 1 (Proof of Theorem 1) *Suppose there exists a PPT adversary $\mathcal{A}$ that attempts to win the Vault Indistinguishability game with a non-negligible advantage. We construct a simulator $\mathcal{B}$ that breaks the IND-CCA security of Σ as follows:*

1. $\mathcal{B}$ acts as the V-IND challenger for $\mathcal{A}$ and as an adversary against the standard IND-CCA challenger Ch_Σ .
2. $\mathcal{A}$ selects two distinct containers $\mathcal{C}_0, \mathcal{C}_1$ and an account $A \notin \tilde{\mathbf{A}}$. $\mathcal{B}$ ensures $|\mathcal{C}_0| = |\mathcal{C}_1| = L$ via the padding rules defined in Section 5.1.
3. $\mathcal{B}$ submits $(\mathcal{C}_0, \mathcal{C}_1)$ to Ch_Σ , receiving a challenge ciphertext $\mathcal{V}^* = \Sigma.\text{Enc}_{K_A}(\mathcal{C}_b)$ for a random bit $b \in \{0, 1\}$.
4. $\mathcal{B}$ sends $\mathcal{V}^*$ to $\mathcal{A}$.
5. If $\mathcal{A}$ queries a decryption oracle for any vault $\mathcal{V} \neq \mathcal{V}^*$, $\mathcal{B}$ forwards this query to Ch_Σ ’s decryption oracle and returns the result to $\mathcal{A}$.
6. $\mathcal{A}$ outputs a guess b' . $\mathcal{B}$ outputs b' as its own guess in the IND-CCA game.

Hence, the advantage of $\mathcal{A}$ in the V-IND game is bounded by the advantage of $\mathcal{B}$ in the IND-CCA game. By the assumption that Σ is IND-CCA secure, we have $\text{Adv}_{\Pi^\dagger}^{v\text{-ind}}(\mathcal{A}) \leq \text{Adv}_{\Sigma}^{\text{ind-cca}}(\mathcal{B}) \leq \text{negl}(\lambda)$.

Proof 2 (Proof of Theorem 2) As per Figure 4, the provider observes a protocol transcript $\text{trans} = \{A, \mathcal{V}_P, \mathcal{V}_D, \sigma\}$ where A is the account ID and σ is provider's signature on $\mathcal{V}_D$. The account A and the provider's current vault $\mathcal{V}_P$ are independent of whether the device is performing a READ or an UPDATE interaction. Here $\mathcal{V}_P = \Sigma.\text{Enc}_{K_A}(\mathcal{C}^{(w)})$ where $\mathcal{C}^{(w)}$ is the w^{th} i.e., the current container stored on the provider's end. Let trans_0 be a transcript of a READ interaction with $\mathcal{V}_D := \mathcal{V}_{D0}$ and trans_1 be a transcript of an UPDATE interaction with $\mathcal{V}_D := \mathcal{V}_{D1}$. For the READ operation, $\mathcal{V}_{D0} = \Sigma.\text{Enc}_{K_A}(\mathcal{C}^{(w)})$ whereas for the UPDATE operation, $\mathcal{V}_{D1} = \Sigma.\text{Enc}_{K_A}(\mathcal{C}^{(w+1)})$.

Σ is a randomized encryption and $\Pi^\dagger$ is V-IND secure which means $\mathcal{V}_P$, $\mathcal{V}_{D0}$ and $\mathcal{V}_{D1}$ are computationally indistinguishable to $\mathcal{A}$ as $\mathcal{A}$ does not possess K_A . Adding to that, σ is a signature on $\mathcal{V}_D$. Since $\mathcal{V}_D$ is indistinguishable, the resulting signature σ does not provide additional distinguishing information.

Finally, the time interval Δt between the provider sending $\mathcal{V}_P$ and receiving $\mathcal{V}_D$ (which is computed by the device in a constant-time T_{const} regardless of the operation type) leaks nothing about the operation type. Combining, we have $\text{Adv}_{\Pi^\dagger}^{\text{it-obl}}(\mathcal{A}) \leq \text{Adv}_{\Pi^\dagger}^{\text{v-ind}}(\mathcal{A}) \leq \text{negl}(\lambda)$.

Proof 3 (Proof of Theorem 3) Let $A \notin \tilde{\mathbf{A}}$ be the account that the covert adversary $\mathcal{A}$ has chosen for a rollback attack, and $\mathcal{D}_A$ is the set of all devices associated to A with $|\mathcal{D}_A| \leq d$. Suppose $\mathcal{A}$ performs a rollback attack by serving an outdated vault $\mathcal{V}^{(j)}$ where $j < n$ and n is the latest vault counter among all devices of $\mathcal{D}_A$. We bound $\text{Adv}_{\Pi^\dagger}^{\text{c-fr}}(\mathcal{A})$ as follows:

1. The EUF-CMA security of $\mathcal{S}$ ensures that some nonempty subset of $\mathcal{D}_A$, let say, $\mathcal{D}_{A,j}$ holds a non-repudiable signature $\sigma^{(k)}$ for a vault state $\mathcal{V}^{(k)}$ with $n \geq k > j$. This constitutes verifiable proof of the provider's commitment to state $\mathcal{V}^{(k)}$.
2. In the security game, $\mathcal{C}$ selects a device $D_i \in \mathcal{D}_A$ uniformly at random. Now, $\mathcal{A}$ is detected under the **Check** procedure if D_i holds a local counter $w_i > j$, i.e., if $D_i \in \mathcal{D}_{A,j}$.
3. Assuming Device Unlinkability, we know that all device interaction transcripts are computationally indistinguishable to $\mathcal{A}$, meaning $\mathcal{A}$ cannot identify if the device D_i that it is interacting with also holds $w_i > j$ or not.
4. Since there are $|\mathcal{D}_{A,j}| \geq 1$ many devices that hold $w_i > j$, the probability of $\mathcal{A}$ successfully interacting with an outdated device is :

$$\text{Adv}_{\Pi^\dagger}^{\text{c-fr}}(\mathcal{A}) \leq 1 - \Pr[D_i \in \mathcal{D}_{A,j}] \leq 1 - \frac{|\mathcal{D}_{A,j}|}{|\mathcal{D}_A|} \leq \left(1 - \frac{1}{d}\right).$$

Proof 4 (Proof of Theorem 4) Suppose there exists a PPT adversary $\mathcal{A}$ that attempts to win the Covert Integrity game by producing a modified vault that decrypts successfully without detection. We construct a simulator $\mathcal{B}$ that breaks the INT-CTXT security of the underlying AE scheme Σ as follows:

1. $\mathcal{B}$ acts as the C-INT challenger for $\mathcal{A}$ and as an adversary against the standard INT-CTXT challenger Ch_Σ .
2. To simulate the legitimate protocol history for an account $A \notin \tilde{\mathbf{A}}$, $\mathcal{B}$ handles $\mathcal{A}$ requests for vault updates. For each legitimate container $\mathcal{C}^{(i)}$ produced during the protocol execution, $\mathcal{B}$ submits $\mathcal{C}^{(i)}$ to its own INT-CTXT encryption oracle (which is $\Sigma.\text{Enc}_{K_A}(\cdot)$) to obtain the corresponding vault $\mathcal{V}^{(i)}$ and provides it to $\mathcal{A}$.
3. $\mathcal{A}$ eventually outputs a modified vault $\mathcal{V}^*$ for an interaction I , such that $\mathcal{V}^* \neq \mathcal{V}^{(j)}$ for any legitimate vault $\mathcal{V}^{(j)}$ previously provided by the simulator.
4. $\mathcal{B}$ submits $\mathcal{V}^*$ to its own INT-CTXT challenger as its forgery attempt.
5. If the user device does not detect the misbehavior, it implies by definition that $\Sigma.\text{Dec}_{K_A}(\mathcal{V}^*) \neq \perp$. In this scenario, $\mathcal{V}^*$ is a valid ciphertext that was never output by the encryption oracle, meaning $\mathcal{B}$ has successfully produced a forgery and wins the INT-CTXT game.

The probability that $\mathcal{A}$ produces an undetected modification is exactly the probability that $\mathcal{B}$ outputs a successful forgery in the INT-CTXT game. By the assumption that Σ is INT-CTXT secure, we have $\text{Adv}_{\Pi^\dagger}^{c\text{-int}}(\mathcal{A}) \leq \text{Adv}_{\Sigma}^{\text{int-ctxt}}(\mathcal{B}) \leq \text{negl}(\lambda)$.

Proof 5 (Proof of Theorem 5) Suppose there exists a PPT adversary $\mathcal{A}$ that can distinguish if two transcripts $\text{trans}_1, \text{trans}_2$ originate from the same device D_i or two different devices D_i, D_j .

1. As per Figure 4, the transcripts contain the tuple $\{A, \mathcal{V}_P, \mathcal{V}_D, \sigma\}$.
2. The account ID A is common to all devices. $\mathcal{V}_P$ is provided by $\mathcal{A}$ itself. σ is a signature on $\mathcal{V}_D$. The only device-generated component is $\mathcal{V}_D := \Sigma.\text{Enc}_{K_A}(\mathcal{C}_D)$ that encrypts the latest container on device D . $\mathcal{C}_D$ may contain metadata and other information about device D .
3. If $\mathcal{A}$ can link sessions, it must be able to distinguish between $\Sigma.\text{Enc}_{K_A}(\mathcal{C}_{D_i})$ and $\Sigma.\text{Enc}_{K_A}(\mathcal{C}_{D_j})$ across sessions over devices D_i and D_j .
4. As demonstrated in the proof of Theorem 1, this capability reduces to breaking the V-IND security of $\Pi^\dagger$.

5. Furthermore, the normalization of execution time to a constant T_{norm} ensures that hardware-specific processing speeds (e.g., a high-end PC vs. a low-power IoT device) are masked.

Therefore, the adversary’s advantage in linking interactions to specific devices is $\text{Adv}_{\Pi^\dagger}^{d\text{-unlk}}(\mathcal{A}) \leq \text{Adv}_{\Pi^\dagger}^{v\text{-ind}}(\mathcal{A}) \leq \text{negl}(\lambda)$.

Conclusion

Although widely deployed, existing password manager solutions fail to consider realistic threat models. Indeed, they assume that providers always act in accordance with their specifications, disregarding scenarios in which a provider may deviate from the protocol when such misbehavior is unlikely to be detected. Even the most recent guidelines published by NIST in August 2024 continue to treat password manager providers as trusted entities.

Our work challenges this trust model by arguing that the presence of a covert adversary is not only realistic but also likely in practice. We further claim that designing password manager solutions resistant to such adversaries is both necessary and technically feasible. To this end, we formalize a set of security properties that such systems should satisfy and propose a concrete solution that provably meets these requirements.

Given the low overhead and the substantial risk reduction, we argue that there is no scientific rationale for excluding covert adversaries from the threat model. We hope that this work will encourage the adoption of more robust, verifiable, and privacy-preserving password manager solutions.

9 Appendix – Practical Considerations Details

This section provides additional depth on the practical considerations we addressed in defining the security properties, designing our password manager protocol, and formally analyzing its security.

9.1 Efficacy of the $1/d$ Deterrence Threshold

In Definition 4.3.3, we define the deterrence threshold for Covert Freshness as $\delta = 1/d$, where d maximizes the number of devices linked to an account. While δ decreases as d increases, we argue that in a covert adversary model, any non-negligible probability of detection serves as a total deterrent.

Unlike a malicious adversary who may accept a risk of discovery to achieve a specific goal, a covert provider utility is tied to its reputation and legal standing. Because the provider produces a non-repudiable digital signature for every vault update, a single successful detection by any of the d devices results in undeniable evidence of misbehavior. In organizational settings where $\text{Cost} \gg \text{Gain}$, the provider cannot risk even a $1/d$ chance of

business termination or legal liability, in particular since it does not know the value of d .

9.2 Timing Side-Channel Mitigation via T_{norm}

To satisfy the property of Device Unlinkability, the protocol must neutralize hardware fingerprinting. Significant variations in processing speed between high-performance workstations and low-power mobile devices allow an adversary to distinguish between clients based on response latency.

By enforcing T_{norm} , we ensure that all client responses are transmitted only after a fixed, standardized duration. This normalization renders fast and slow hardware computationally indistinguishable to the provider in the time domain. In a production environment, T_{norm} can be calibrated to match the worst-case processing time of common modern hardware, ensuring that the introduced latency remains imperceptible to the end-user while providing robust protection against timing-based identification.

9.3 Decoupling Encryption from Master Passwords

Traditional password managers typically derive encryption keys from user-defined master passwords, leaving them vulnerable to large-scale offline cracking by the provider. Our architecture utilizes a randomly generated account key that is never transmitted to the provider.

The introduction of a “*one-time*” out-of-band key exchange (utilizing numeric codes or visual identifiers) to link new devices is a necessary shift to move the encryption key out of the provider’s reach. By requiring this manual pairing step, the security of the vault is no longer bottlenecked by the entropy of a user’s password, effectively eliminating the master password as a systemic point of failure in the cloud infrastructure.

9.4 Consistency Guarantees for Asynchronous Settings

In asynchronous environments where devices frequently remain offline, ensuring data consistency is typically a complex challenge. However, our model leverages the Covert Freshness property to force the provider into honest behavior.

Because the provider cannot predict which device will next initiate a synchronization request, and because any attempt to serve stale data carries a non-trivial risk of producing verifiable evidence of fraud, the provider is economically and technically compelled to serve the latest vault version. Consequently, our architecture provides a form of intrinsic consistency; the threat of detection ensures that the provider acts as a reliable synchronization relay, allowing returning devices to seamlessly reconcile their state with the most recent global update.

10 Appendix – Formal Security Definitions

We recall below the standard game-based primitive definitions.

Definition 2 (IND-CCA Security) *An AE scheme Σ is IND-CCA secure if for any PPT adversary $\mathcal{A}$, the advantage in the following experiment is $\text{negl}(\lambda)$:*

1. *Ch samples a random and secret $K \leftarrow \Sigma.\text{KGen}(1^\lambda)$.*
2. *$\mathcal{A}^{\Sigma.\text{Enc}_K(\cdot), \Sigma.\text{Dec}_K(\cdot)}$ provides challenge pair (M_0, M_1) such that $|M_0| = |M_1|$.*
3. *Ch samples $b \xleftarrow{\$} \{0, 1\}$ and computes $C^* \leftarrow \Sigma.\text{Enc}_K(M_b)$.*
4. *$\mathcal{A}^{\Sigma.\text{Enc}_K(\cdot), \Sigma.\text{Dec}_K(\cdot)}$ who is provided with C^* and is not allowed to query $\Sigma.\text{Dec}_K(C^*)$ provides a bit b' .*
5. *$\mathcal{A}$ wins if $b' = b$ with probability $> 1/2$.*

Thus, $\mathcal{A}$'s advantage in breaking the IND-CCA security of Σ is:

$$\text{Adv}_{\Sigma}^{\text{ind-cca}}(\mathcal{A}) := |\Pr[b' = b] - 1/2|$$

and say Σ is IND-CCA-secure if for all efficient adversaries $\mathcal{A}$, $\text{Adv}_{\Sigma}^{\text{ind-cca}}(\mathcal{A}) \leq \text{negl}(\lambda)$.

Definition 3 (INT-CTXT Security) *An AE scheme Σ provides Cipher-text Integrity i.e., INT-CTXT security if for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}^{\Sigma.\text{Enc}_K(\cdot), \Sigma.\text{Dec}_K(\cdot)}$ outputs a pair (C) such that $\Sigma.\text{Dec}_K(C) \neq \perp$ (where C was not previously output by the encryption oracle under associated data AD) is $\text{negl}(\lambda)$. In other words,*

$$\begin{aligned} \text{Adv}_{\Sigma}^{\text{int-ctxt}}(\mathcal{A}) &:= \Pr[\mathcal{A}^{\Sigma.\text{Enc}_K(\cdot), \Sigma.\text{Dec}_K(\cdot)} \Rightarrow C \mid \Sigma.\text{Dec}_K(C) \neq \perp \\ &\quad \wedge C \text{ is not an output of } \text{Enc}_K] \leq \text{negl}(\lambda). \end{aligned}$$

Remark. AE schemes in literature are mostly nonce-based and target the *nonce-based AE* (nAE) security [28]. We note that nAE security is basically IND-CCA+INT-CTXT when nonces are assumed to be unique for every encryption call (see [9, 4]).

Definition 4 (EUF-CMA Security) *A digital signature scheme $\mathcal{S}$ is said to be Existentially Unforgeable under Chosen Message Attack (EUF-CMA secure) if for any PPT adversary $\mathcal{A}$ (who has access to pk), the probability that $\mathcal{A}^{\mathcal{S}.\text{Sign}_{sk}(\cdot)}$ outputs (M, σ) such that $\mathcal{S}.\text{Vrfy}_{pk}(M, \sigma) = 1$ (where M was never queried to the signing oracle) is $\text{negl}(\lambda)$. In other words:*

$$\begin{aligned} \text{Adv}_{\mathcal{S}}^{\text{euf-cma}}(\mathcal{A}) &:= \Pr[\mathcal{A}^{\mathcal{S}.\text{Sign}_{sk}(\cdot)} \Rightarrow (M, \sigma) \mid \mathcal{S}.\text{Vrfy}_{pk}(M, \sigma) = 1 \\ &\quad \wedge M \text{ is not signed before}] \leq \text{negl}(\lambda). \end{aligned}$$

References

- [1] 1Password. 1password security design. <https://1password.com/files/1password-white-paper.pdf>, 2021. Accessed: 2026-01-09.
- [2] Nora Alkaldi and Karen Renaud. Encouraging password manager adoption by meeting adopter self-determination needs. In *52nd Hawaii International Conference on System Sciences, HICSS*, 2019.
- [3] Yonatan Aumann and Yehuda Lindell. Security against covert adversaries: Efficient protocols for realistic adversaries. In *Theory of Cryptography, 4th Theory of Cryptography Conference, TCC*, Lecture Notes in Computer Science. Springer, 2007.
- [4] Mihir Bellare and Chanathip Namprempre. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2000.
- [5] Bitwarden. World password day survey 2025. <https://bitwarden.com/resources/world-password-day-2024/>, 2025. Accessed: 2026-01-09.
- [6] Bitwarden Inc. Bitwarden security whitepaper. <https://bitwarden.com/help/bitwarden-security-white-paper/>, Oct 2025. Accessed: 2026-01-09.
- [7] Hristo Bojinov, Elie Bursztein, Xavier Boyen, and Dan Boneh. Kamouflage: Loss-resistant password management. In *Computer Security - ESORICS*. Springer, 2010.
- [8] Faysal Boukayoua, Bart De Decker, and Vincent Naessens. A keyboard that manages your passwords in Android. In *2014 International Conference on Privacy and Security in Mobile Systems (PRISMS)*, 2014.
- [9] A. A. Bozhko and CryptoPro. Properties of AEAD Algorithms. IRTF Draft, CRYPTO Forum (2024). URL: <https://www.ietf.org/archive/id/draft-irtf-cfrg-aead-properties-07.html#name-conventional-properties>.
- [10] BSI. Mindeststandard des bsi für webbrowser-version 3.0 vom 19.02.2024. <https://www.bsi.bund.de/dok/MST-Browser>, Feb 2024. Accessed: 2026-01-09.
- [11] Rahul Chatterjee, Joseph Bonneau, Ari Juels, and Thomas Ristenpart. Cracking-resistant password vaults using natural language encoders. In *IEEE Symposium on Security and Privacy, SP*, 2015.

- [12] Sunil Chaudhary, Tiina Schafeitel-Tähtinen, Marko Helenius, and Eleni Berki. Usability, security and trust in password managers: A quest for user-centric properties and features. *Computer Science Review*, 2019.
- [13] Sonia Chiasson, Paul C van Oorschot, and Robert Biddle. A usability study and critique of two password managers. In *USENIX Security Symposium*, 2006.
- [14] Mark Ciampa, Mark Revels, and John Enamait. Online versus local password management applications: An analysis of user training and reactions. *Journal of Applied Security Research*, 2011.
- [15] Dashlane Inc. Dashlane security white paper. <https://www.dashlane.com/download/whitepaper-en.pdf>, Jan 2025. Accessed: 2026-01-09.
- [16] Danny Dolev and Andrew Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 1983.
- [17] Michael Fagan, Yusuf Albayram, Mohammad Maifi Hasan Khan, and Ross Buck. An investigation into users’ considerations towards using password managers. *Hum. centric Comput. Inf. Sci.*, 2017.
- [18] Sascha Fahl, Marian Harbach, Marten Oltrogge, Thomas Muders, and Matthew Smith. Hey, you, get off of my clipboard. In *International Conference on Financial Cryptography and Data Security*. Springer, 2013.
- [19] Paolo Gasti and Kasper Bonne Rasmussen. On the security of password manager database formats. In *Computer Security - ESORICS*. Springer, 2012.
- [20] Google. Manage encryption for passwords in your Google account. <https://support.google.com/accounts/answer/11350823?hl=en>, 2026. Accessed: 2026-01-09.
- [21] Nicolas Huaman, Sabrina Amft, Marten Oltrogge, Yasemin Acar, and Sascha Fahl. They would do better if they worked together: The case of interaction problems between password managers and websites. In *IEEE Symposium on Security and Privacy, SP*, 2021.
- [22] Iulia Ion, Rob Reeder, and Sunny Consolvo. “... no one can hack my mind:” comparing expert and non-expert security practices. In *Eleventh Symposium On Usable Privacy and Security (SOUPS 2015)*, 2015.
- [23] LastPass. Lastpass technical whitepaper. <https://assets.cdngetgo.com/da/ce/d211c1074dea84e06cad6f2c8b8e/lastpass-technical-whitepaper.pdf>, Dec 2025. Accessed: 2026-01-09.

- [24] Tao Liu, Gowri Ramachandran, and Raja Jurdak. Towards quantum resilient iot: A backward-compatible approach to secure ble key exchange against quantum threats. In *2024 IEEE/ACM Ninth International Conference on Internet-of-Things Design and Implementation (IoTDI)*, pages 170–180, 2024.
- [25] Sanam Ghorbani Lyastani, Michael Schilling, Sascha Fahl, Michael Backes, and Sven Bugiel. Better managed than memorized? studying the impact of managers on password strength and reuse. In *27th USENIX security symposium*, 2018.
- [26] Alex McOmie. 65% of people don’t trust password managers despite 60% experiencing a data breach. <https://www.passwordmanager.com/password-manager-trust-survey/>, 2023. Accessed: 2026-01-09.
- [27] Zaur Molotnikov. KeePassXC application security review. <https://keepassxc.org/assets/pdf/KeePassXC-Review-V1-Molotnikov.pdf>, 2023. Accessed: 2026-01-13.
- [28] Chanathip Namprempre, Phillip Rogaway, and Thomas Shrimpton. Reconsidering generic composition. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 257–274. Springer, 2014.
- [29] National Institute of Standards and Technology. Digital Identity Guidelines: Authentication and Lifecycle Management. Technical Report NIST SP 800-63B, U.S. Department of Commerce, 2024. Accessed: 2026-01-09. URL: <https://doi.org/10.6028/NIST.SP.800-63b>.
- [30] NordPass. NordPass Business: The modern password management solution. <https://assets.nordpass.com/nordpass-business-whitepaper.pdf>, Oct 2025. Accessed: 2026-01-09.
- [31] NortonLifeLock Inc. Norton Password Manager. <https://us.norton.com/feature/password-manager>, 2026. Accessed: 2026-01-09.
- [32] Sean Oesch and Scott Ruoti. That was then, this is now: A security evaluation of password generation, storage, and autofill in browser-based password managers. In *USENIX Security*, 2020.
- [33] Sean Oesch, Scott Ruoti, James Simmons, and Anuj Gautam. “it basically started using me:” an observational study of password manager usage. In *CHI ’22: Conference on Human Factors in Computing Systems*, 2022.

- [34] Hirak Ray, Flynn Wolf, Ravi Kuber, and Adam J. Aviv. Why older adults (don't) use password managers. In *30th USENIX Security Symposium*, 2021.
- [35] RoboForm Inc. Roboform security overview whitepaper. https://www.roboform.com/pdf/RoboForm_Security_White_Paper.pdf, Feb 2023. Accessed: 2026-01-09.
- [36] Security.org. 2024 password manager industry report and statistics. <https://www.security.org/digital-safety/password-manager-annual-report/>, Jul 2025. Accessed: 2026-01-09.
- [37] Maliheh Shirvanian, Christopher Robert Price, Mohammed Jubur, Nitesh Saxena, Stanislaw Jarecki, and Hugo Krawczyk. A hidden-password online password manager. In *SAC '21: The 36th ACM/SIGAPP Symposium on Applied Computing*, 2021.
- [38] David Silver, Suman Jana, Dan Boneh, Eric Yawei Chen, and Collin Jackson. Password managers: Attacks and defenses. In *Proceedings of the 23rd USENIX Security Symposium*, 2014.
- [39] Xing Wang, Zhen Han, and Dawei Zhang. IDKeeper: A web password manager with roaming capability based on USB key. In *International Conference on Industrial Control and Electronics Engineering*, 2012.
- [40] Bian Yang, Huiguang Chu, Guoqiang Li, Slobodan Petrovic, and Christoph Busch. Cloud password manager using privacy-preserved biometrics. In *IEEE International Conference on Cloud Engineering*, 2014.
- [41] Shikun Zhang, Sarah Pearman, Lujo Bauer, and Nicolas Christin. Why people (don't) use password managers effectively. In *Fifteenth Symposium on Usable Privacy and Security, SOUPS*. USENIX Association, 2019.